

Ask, Don't Guess: Learning When to Clarify in Situated Instruction Following

Anonymous authors

Paper under double-blind review

Abstract

Language agents deployed in situated environments must decide not only what an instruction might mean, but whether it is useful to act on that interpretation. Existing ambiguity and clarification evaluations often frame this as a static language-understanding problem: detect ambiguity and ask a question. We argue that clarification is instead a situated decision under uncertainty, interaction cost, and task consequences. We introduce CLARIFY-TO-ACT, a procedurally generated benchmark in which an agent receives a structured scene and a user instruction, then must either act immediately or ask one clarifying question before acting. The environment provides deterministic rewards based on final task success minus clarification cost, enabling cheap learning from interaction without human labels or model weight updates. In a 100-episode stratified API evaluation with GPT-4.1-mini, a prompted ask-when-needed baseline reaches 0.632 net utility and misses 58.3% of oracle clarification cases, while an expected-communicative-utility policy reaches 0.976 net utility with 100% task success and no missed or unnecessary clarifications.

1 Introduction

Situated language agents frequently face instructions that are underspecified relative to the action space. A household assistant asked to “bring the red mug” may see two red mugs. A file-management agent asked to “delete the old draft” may see two plausible drafts with very different consequences. In these cases, the agent must decide whether to act immediately or ask a clarifying question.

The usual framing asks whether the user query is ambiguous (Zhang et al., 2024; Kuhn et al., 2022). That framing is incomplete. Some ambiguous instructions are resolved by the environment: if only one cup is reachable on the active workspace, asking wastes interaction. Some underspecified instructions are harmless because all choices are equivalent: “move a spare chair to the table” does not require identifying a particular spare chair. Other cases are high risk: even a likely interpretation of “delete the old file” may warrant clarification if the wrong action is costly.

We study clarification as a situated decision problem. The relevant question is not simply whether language admits multiple interpretations, but whether the expected value of clarification exceeds its interaction cost. This connects clarification to situated instruction following (Min et al., 2024) and value-of-information views of questions (Rao & Daumé, 2018), while keeping the benchmark lightweight and deterministic.

This paper makes three contributions. First, we introduce CLARIFY-TO-ACT, a procedural benchmark for ask-or-act decisions in structured scenes. Second, we define an expected communicative utility objective that provides automatic oracle labels and task-level net utility. Third, we evaluate deterministic baselines, a lightweight learned controller, and API-based language-agent policies. Expected-utility calibration improves net utility by reducing both guessing and unnecessary clarification.

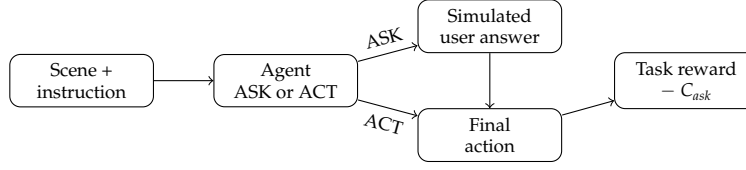


Figure 1: CLARIFY-TO-ACT interaction protocol. The agent either acts immediately or asks one clarifying question, then receives deterministic task reward minus interaction cost.

Category	Example instruction	Oracle ask	Diagnostic contrast
Referential	Bring the black box	1.00	Matching objects, wrong target matters
Context-resolved	Pass the folder I’m using	0.00	Context or salience resolves intent
Equivalent outcome	Move a spare folder	0.00	Any target in success class works
Risk-sensitive	Delete the old draft	1.00	Wrong action has high cost
Preference/social	Put my cup away	0.50	Owner visible vs. hidden

Table 1: Benchmark categories. Each category has 280 episodes in the canonical dataset; the oracle ask rate follows expected communicative utility.

2 Clarify-to-Act

Each episode contains a scene, a user instruction, candidate intents, a hidden user intent, a simulated user, and a deterministic reward. The agent observes the scene and instruction and outputs either:

$$\{\text{type: ACT, action: } a, \text{target_id: } o\}$$

or:

$$\{\text{type: ASK, question: } q\}.$$

If it asks, the simulated user returns one answer from the hidden intent and the agent must then act. Episodes are at most two turns.

Diagnostic categories. Table 1 summarizes the five diagnostic categories. The current dataset contains 1,400 episodes: 600 train, 200 dev, 400 test, and 200 OOD test episodes. Splits are balanced across categories and the overall oracle ask rate is 0.50.

Reward and oracle. The reward is

$$R = \mathbb{1}[\text{success}] - C_{ask}\mathbb{1}[\text{asked}] - C_{wrong}\mathbb{1}[\text{wrong}].$$

Success is defined by matching the hidden intent’s success-equivalence class, so interchangeable actions can succeed without arbitrary target identity matches. Let $p_{\max}$ be the probability mass of the best success-equivalence class. We compute

$$EU(\text{act}) = p_{\max} - (1 - p_{\max})C_{wrong}, \quad EU(\text{ask}) = 1 - C_{ask}.$$

The oracle asks when $EU(\text{ask}) > EU(\text{act}) + \epsilon$.

Leakage controls. API smoke tests exposed possible leakage in the preference/social category. The final benchmark uses neutral target IDs, redacts hidden object owners from API prompts, gives hidden-owner objects neutral state labels, and includes current_user only as user identity. Visible-owner cases remain resolvable; hidden-owner cases require clarification.

3 Methods

DirectAct. The agent acts immediately and never asks.

AskAlways and raw ambiguity. AskAlways always asks before acting. Raw ambiguity asks whenever more than one candidate target exists.

Method	Ask/act signal	Diagnostic role
DirectAct	Never asks	Guessing under ambiguity and risk
AskAlways / raw ambiguity	Always ask / target count	Over-clarification and ambiguity-only behavior
Prompted Ask-Needed	Ask-or-act prompt	Strong simple prompting baseline
CoT Ask-Needed	Private-reasoning prompt	Reasoning-only calibration test
API ECU	Model candidates + expected utility	Main utility-calibrated policy
Threshold / controller	Tuned margin / logistic model	Interaction-supervised calibration

Table 2: Method summary. The key comparison is not only whether a method detects ambiguity, but whether its ask/act decision is calibrated to expected task utility.

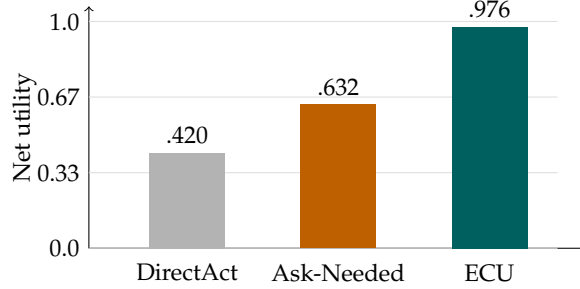


Figure 2: Main API result on 100 stratified test episodes. ECU substantially improves net utility over direct action and prompted ask-when-needed.

65 **Prompted Ask-Needed.** An API language model receives a prompt instructing it to ask
 66 only when acting could plausibly fail. This is the strongest simple prompting baseline and
 67 is related to recent language-agent clarification work (Qian et al., 2024).

68 **Expected communicative utility.** The ECU policy asks the model to propose candidate
 69 interpretations with probabilities and metadata. Python computes expected utility from
 70 the candidates, ask cost, and wrong-action cost. If the margin favors asking, the model
 71 generates one clarifying question; otherwise the agent acts on the best candidate. The final
 72 API ECU policy uses an ask margin of 0.075, visibility redaction for hidden-owner cases,
 73 and a conservative equivalence guard so the model can only collapse candidates into one
 74 success class when the instruction contains equivalence cues such as “spare” or “any.”

75 **Learned controller.** The offline learned controller is a lightweight logistic ask/act model
 76 trained from automatic oracle labels. It uses features such as number of candidates, number
 77 of success classes, top prior, entropy, ask cost, wrong-action cost, salience gap, risk, context
 78 resolution, equivalence, and expected-utility margin.

79 4 Experiments

80 We report offline benchmark evaluation, API evaluation, and cost sensitivity. Offline
 81 evaluation uses the full 400-episode test split and 200-episode OOD split. API evaluation
 82 uses GPT-4.1-mini on a 100-episode stratified subset with 20 episodes per category. We
 83 additionally construct a 50-episode stress set by paraphrasing test instructions and shifting
 84 simulated user answers to terser location/owner-style replies. As a small model-coverage
 85 sanity check, we also run GPT-4.1-nano on 25 stratified test episodes. Calls are cached; the
 86 final main cache contains 914 responses, 271,208 input tokens, and 49,117 output tokens.

87 **Reproducibility.** The artifact package includes a generated reproducibility report with
 88 dataset and result hashes, recomputed metrics, API cache token totals, and exact commands.
 89 It also includes automated claim-verification and simulated-user answer audits. The final
 90 API evaluation can be replayed in cache-only mode, which fails on any cache miss rather
 91 than making a network call.

Method	N	Utility	Success	Ask	Missed	Unnec.
DirectAct	100	0.420	0.770	0.000	1.000	0.000
Ask-Needed	100	0.632	0.880	0.370	0.583	0.327
API ECU	100	0.976	1.000	0.480	0.000	0.000

Table 3: API evaluation with GPT-4.1-mini. Utility is the primary metric.

Category	Direct	Ask-Needed	ECU	ECU Ask	Oracle Ask
Context-resolved	1.000	0.993	1.000	0.000	0.000
Preference/social	0.400	0.400	0.980	0.400	0.400
Equivalent-outcome	1.000	0.920	1.000	0.000	0.000
Referential	-0.100	0.857	0.950	1.000	1.000
Risk-sensitive	-0.200	-0.008	0.950	1.000	1.000

Table 4: API net utility by category. ECU asks when clarification matters and avoids asking when context or outcome equivalence resolves the instruction.

5 Results

Table 3 and Figure 2 show the main API result. DirectAct fails because it never asks, producing low success in referential and risk-sensitive cases. Prompted Ask-Needed improves over DirectAct but remains poorly calibrated: it misses 58.3% of oracle clarification cases and asks unnecessarily in 32.7% of oracle-act cases. A private-reasoning variant of Ask-Needed does not improve net utility (0.632) and still misses 60.4% of oracle clarification cases. API ECU reaches 0.976 net utility and 100% success, with no missed or unnecessary clarifications in this stratified evaluation. Because all policies run on the same episodes, paired bootstrap deltas are the appropriate comparison: ECU exceeds Ask-Needed by 0.343 net utility with 95% paired CI [0.168, 0.559], and exceeds the private-reasoning variant by 0.344 [0.171, 0.564]. Conditional on asking, both Ask-Needed and ECU successfully ground the simulated answer in the main set; the gap is ask timing. ECU has ask precision/recall/post-answer success of 1.000/1.000/1.000, while Ask-Needed has 0.541/0.417/1.000.

Table 4 shows that the ECU advantage is not driven by a single category. Prompted Ask-Needed performs well on referential ambiguity but over-asks equivalent-outcome cases and misses most risk-sensitive cases. ECU asks in all referential and risk-sensitive cases, avoids all context-resolved and equivalent-outcome cases, and matches the oracle ask rate in preference/social cases. In a no-API subset-stability diagnostic, ECU-Ask-Needed remains positive after omitting any one category (minimum +0.190) or any one episode (minimum +0.307), and a category-stratified bootstrap gives 95% CI [0.183, 0.541].

Ambiguity is not enough. A no-API diagnostic isolates the central distinction from raw ambiguity detection. All 400 canonical test episodes have multiple candidate interpretations, but exactly 200 are oracle-act cases and 200 are oracle-ask cases. A surface-ambiguity policy therefore asks on every episode, reaching 0.920 utility with unnecessary clarification rate 1.000. A supervised uncertainty-only controller trained only on candidate count, top prior, and prior entropy reaches 0.900 utility, but still misses 7.0% of oracle-ask cases and asks unnecessarily in 40.0% of oracle-act cases. ECU and the full learned controller reach 0.958 utility while eliminating both missed and unnecessary clarification on the same offline test split. A situated contrast diagnostic makes the same point at the slice level. Among two-candidate bring episodes, all 80 context-resolved cases are oracle-act while all 80 referential cases are oracle-ask. For put-away preference episodes, visible ownership gives oracle-act on 40/40 cases, while hidden ownership gives oracle-ask on 40/40. High-entropy equivalent-outcome cases are oracle-act because all candidates share one success class, whereas high-top-prior risk-sensitive cases are oracle-ask because the wrong-action cost is high.

Utility-margin calibration. We also bin episodes by the oracle expected-utility margin $EU(ask) - EU(act)$. On the main API set, ECU asks on 0/40 act-preferred episodes and 48/48 ask-preferred episodes. Prompted Ask-Needed asks on 17/40 act-preferred episodes and only 20/48 ask-preferred episodes. On the paraphrase/style stress set, ECU again asks on 0/20 act-preferred and 23/23 ask-preferred episodes, while Ask-Needed asks on 7/20 and 12/23. Thus the prompted baseline is not calibrated to value of information: it asks at similar rates when asking is harmful and when asking is useful. As an internal cached-row diagnostic, API ECU’s model-derived candidate-margin threshold agrees with the oracle ask label on 99/100 main API rows; after the effective context override, final ask/oracle agreement is 100/100. This supports that the API-side candidate scoring is aligned on the evaluated subset, but it is not an independent benchmark.

A cached decision ablation isolates the role of the equivalence guard. Replaying the same model candidate interpretations, accepting every model-declared equivalence flag drops utility to 0.745 and misses 37.5% of oracle-ask cases. Never collapsing equivalent candidates keeps success at 100% but raises unnecessary clarification to 38.5%. The conservative guard is therefore needed in both directions: it prevents unsafe over-collapsing in true ambiguity while still allowing action in equivalent-outcome cases.

On the full 400-episode offline test set, ECU and the learned controller reach 0.958 net utility, compared with 0.938 for the prompted heuristic, 0.920 for AskAlways, and 0.498 for DirectAct. On the 200-episode OOD split, ECU reaches 0.975 utility versus 0.955 for the prompted heuristic. The OOD split preserves all five diagnostic categories and shifts object types where supported by the generator; 102 OOD episodes contain at least one held-out object type. On this held-out-object slice, ECU and the learned controller reach 0.975 utility and 100% success, while keeping ask rate at 0.500. The learned controller is interpretable: its largest positive weights are expected-utility margin, risk, and entropy, while context resolution, equivalence, and ask cost push toward acting.

We also run a no-API held-out ambiguity-mix diagnostic. Train/dev/test episodes contain only referential, context-resolved, and equivalent-outcome cases; the held-out split contains risk-sensitive and preference/social cases. ECU transfers with 0.962 held-out utility, nearly unchanged from 0.963 on the seen-category test split. The tuned threshold and learned controller reach 0.950 but ask on all held-out episodes, over-clarifying visible preference/social cases. This is a useful boundary on the learning claim: the controller is transparent and effective when the diagnostic categories are represented, while the expected-utility rule is more stable under this category shift.

As an external query-level sanity check, we map CLAMBER’s `require_clarification` field to an ASK label (Zhang et al., 2024). The benchmark’s provided `predict_ambiguous` field has 0.284 recall and 0.716 missed-clarification rate against that label. This is not a situated task-utility result, but it supports the motivation that ambiguity prediction alone can under-identify clarification needs.

Paraphrase and answer-style stress. On the 50-episode API stress set, API ECU reaches 0.977 net utility and 100% success. Prompted Ask-Needed reaches 0.814 utility, misses 47.8% of oracle clarification cases, and asks unnecessarily in 25.9% of oracle-act cases. DirectAct reaches 0.320 utility. The paired ECU–Ask-Needed difference is +0.163 with 95% CI [0.040, 0.321]. This small stress test does not prove broad linguistic robustness, but it checks that the main ask/act calibration result is not limited to the original instruction and answer templates. In the auxiliary 25-episode GPT-4.1-nano check, ECU reaches 0.722 utility versus 0.098 for Ask-Needed and 0.040 for DirectAct, with paired ECU–Ask-Needed difference +0.624 [0.074, 1.254]. This supports the direction of the result under a weaker model, but it remains too small to substitute for a broad model sweep.

Cost sensitivity. When ask cost increases from 0.01 to 0.35 with wrong-action cost fixed at 1.0, ECU ask rate drops from 0.800 to 0.455. The prompted heuristic keeps ask rate fixed at 0.700 and loses utility. When wrong-action cost rises from 0.2 to 3.0 with ask cost fixed at 0.05, DirectAct utility falls from 0.751 to 0.170, while ECU remains near 0.96. We also re-score the fixed cached API outputs under alternate global costs without rerunning the

Failure mode	Observed baseline behavior	ECU decision reason
Referential guessing	"Bring me the black box": DirectAct chose one matching box without asking and failed.	Candidate success classes differ, so asking has higher expected utility.
Risk blindness	"Delete the old draft": Ask-Needed acted immediately; lucky success still violates the ask oracle.	High wrong-action cost makes clarification valuable even when one target seems likely.
Preference blindness	"Put my box away": the model acted with ownership hidden and selected the wrong box.	Hidden owner information prevents resolving "my" from the scene alone.
Equivalence blindness	"Move a spare folder to the table": Ask-Needed asked which spare folder, paying an unnecessary cost.	All spare folders share the same success class, so acting is sufficient.
Context overask	"Bring me the folder": Ask-Needed asked even though the active workspace identified the target.	Context-resolved salience makes immediate action utility-dominant.

Table 5: Representative first-turn calibration failures. The examples illustrate why ambiguity detection alone is insufficient: the right decision depends on outcome equivalence, context, and stakes.

model. ECU retains a positive paired utility delta over Ask-Needed in all tested settings: +0.201 to +0.239 across the ask-cost sweep and +0.138 to +0.475 across the wrong-action-cost sweep, with all paired bootstrap lower bounds above zero (minimum +0.070). This does not test adaptive retuning, but it shows the main API advantage is not tied to one narrow reward parameterization.

Author audit. We audited 100 stratified test scenarios and 100 sampled API clarification questions. All 100 scenario labels were marked coherent with the visible scene, hidden target, and utility oracle. For clarification questions, 73 of 100 were marked ok, 19 minor issue, and 8 bad question. All audited ECU questions for oracle-ask cases were natural and diagnostic. The question issues were expected baseline failures: prompted Ask-Needed asked natural but unnecessary questions, or extra non-diagnostic table/preference questions, in oracle-act equivalent-outcome and context-resolved cases.

6 Qualitative Analysis

We label a failure event whenever the final action fails or the first-turn ask/act decision disagrees with the utility oracle. This counts successful but unnecessary questions and lucky unsafe guesses as calibration failures. On the main API set plus the private-reasoning baseline, DirectAct has 48 failure events, Ask-Needed has 45, and Ask-Needed with private reasoning has 47; API ECU has 0. The largest failure modes are risk blindness (57 events), equivalence blindness (33), and referential guessing (25).

Prompted Ask-Needed exhibits two recurring failure modes. First, it over-clarifies equivalent-outcome instructions. For example, when asked to move a spare folder to the table, it often asks which spare folder the user means even though any spare folder succeeds. Second, it under-clarifies high-risk actions. In several delete-file cases, the prompt baseline acts immediately despite high wrong-action cost. ECU avoids these failures because its decision rule explicitly compares the value of asking to the value of acting.

7 Limitations

CLARIFY-TO-ACT is a synthetic text/JSON environment, not a physical simulator. This isolates ambiguity, context, equivalence, ask cost, and wrong-action cost in a controlled setting, but does not test perception, long-horizon planning, or real human interaction. The simulated user answers deterministically from the hidden intent; a released audit verifies that these answers identify the hidden success class from visible scene fields in 1233/1233 generated oracle-ask cases and 184/184 actual API asked rows, but this is not a human-response study. The headline API evaluation uses one mini model, a 100-episode main subset, and a 50-episode paraphrase/style stress set; the 25-episode nano check is only an auxiliary sanity check. Broader model comparisons should be added only after the core

paper framing is fixed. The learned controller is lightweight and interpretable rather than a model-weight training result.

8 Related Work

Clarification has been studied in question answering, dialogue, and language-agent settings. CLAMBER evaluates whether language models identify and clarify ambiguous information needs (Zhang et al., 2024). CLAM studies selective clarification for ambiguous questions (Kuhn et al., 2022), and value-of-information approaches rank clarification questions by expected utility (Rao & Daumé, 2018). Recent task-oriented dialogue work studies referential ambiguity and clarification request types (Madge et al., 2025). Situated instruction following emphasizes that instructions are grounded in context, environment state, and action consequences (Min et al., 2024). Our core distinction is that clarification is not treated as a property of language alone: the same sentence may call for asking or acting depending on context, equivalence, and stakes.

9 Conclusion

Clarification should be evaluated as a situated decision under uncertainty, interaction cost, and task consequences. CLARIFY-TO-ACT provides a controlled benchmark for this decision. Across offline and API evaluations, expected communicative utility improves net task utility by asking when clarification changes the expected outcome and acting when it does not. These results suggest that pragmatic clarification is best studied as utility-calibrated interaction policy rather than ambiguity detection alone.

References

- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. CLAM: Selective clarification for ambiguous questions with generative language models. *arXiv preprint arXiv:2212.07769*, 2022. doi: 10.48550/arXiv.2212.07769. URL <https://arxiv.org/abs/2212.07769>.
- Chris Madge, Matthew Purver, and Massimo Poesio. Referential ambiguity and clarification requests: Comparing human and LLM behaviour. In *Proceedings of the Eighth Workshop on Computational Models of Reference, Anaphora and Coreference*, pp. 1–11, 2025. doi: 10.18653/v1/2025.crac-1.1. URL <https://aclanthology.org/2025.crac-1.1/>.
- So Yeon Min, Xavi Puig, Devendra Singh Chaplot, Tsung-Yen Yang, Akshara Rai, Priyam Parashar, Ruslan Salakhutdinov, Yonatan Bisk, and Roozbeh Mottaghi. Situated instruction following. *arXiv preprint arXiv:2407.12061*, 2024. doi: 10.48550/arXiv.2407.12061. URL <https://arxiv.org/abs/2407.12061>.
- Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Zhong Zhang, Jie Zhou, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Tell me more! towards implicit user intention understanding of language model driven agents. *arXiv preprint arXiv:2402.09205*, 2024. doi: 10.48550/arXiv.2402.09205. URL <https://arxiv.org/abs/2402.09205>.
- Sudha Rao and Hal Daumé, III. Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information. *arXiv preprint arXiv:1805.04655*, 2018. URL <https://arxiv.org/abs/1805.04655>.
- Tong Zhang, Peixin Qin, Yang Deng, Chen Huang, Wenqiang Lei, Junhong Liu, Dingnan Jin, Hongru Liang, and Tat-Seng Chua. CLAMBER: A benchmark of identifying and clarifying ambiguous information needs in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 10746–10766, 2024. doi: 10.18653/v1/2024.acl-long.578. URL <https://aclanthology.org/2024.acl-long.578/>.